

Postfix SMTP Relay with Gmail OAuth2 and DKIM Signing

Installation Guide for PCLinuxOS / RPM-based Systems

May 2026

Overview

This guide walks through setting up a Postfix SMTP relay on Linux that accepts email from a local mail server (XMail, however it should work for any mail server) and delivers it to Gmail using OAuth2 authentication, with outbound DKIM signing. This setup is necessary because the mail server does not support neither OAuth2 nor DKIM signing, and Google deprecated plain password authentication for SMTP.

Architecture

The mail flow is as follows:

- XMail sends outbound email to Postfix on port 2525 (localhost)
- Postfix passes the message through the DKIM content filter
- The DKIM signing script signs the message and resubmits to Postfix on port 10026
- Postfix relays the signed message to Gmail on port 587 using OAuth2 authentication
- Gmail delivers the email

Port assignments:

- 2525 — inbound from XMail to Postfix (port 25 is taken by XMail)
- 10026 — re-injection port after DKIM signing (already defined in default master.cf)
- 587 — outbound from Postfix to Gmail SMTP

Phase 1 — Create System Users and Groups

On PCLinuxOS the Postfix installer does not create the required system users. Also `/etc/login.defs` may be missing. Create it first:

```
cat > /etc/login.defs << 'EOF'
MAIL_DIR          /var/spool/mail
PASS_MAX_DAYS     99999
PASS_MIN_DAYS     0
PASS_WARN_AGE     7
UID_MIN           1000
UID_MAX           60000
SYS_UID_MIN       201
SYS_UID_MAX       999
GID_MIN           1000
GID_MAX           60000
SYS_GID_MIN       201
SYS_GID_MAX       999
EOF
```

```
CREATE_HOME    yes
UMASK          077
USERGROUPS_ENAB yes
ENCRYPT_METHOD  SHA512
EOF
```

Create the groups and users directly in the system files:

```
# Groups
echo 'postfix:x:89:' >> /etc/group
echo 'postdrop:x:90:' >> /etc/group
echo 'dkim:x:91:' >> /etc/group

# Users
echo 'postfix:x:89:89:Postfix Mail Server:/var/spool/postfix:/sbin/nologin' >> /etc/passwd
echo 'dkim:x:91:91:Postfix DKIM Signer:/var/spool/postfix:/sbin/nologin' >> /etc/passwd

# Shadow entries
echo 'postfix:!:19000:0:99999:7:::' >> /etc/shadow
echo 'dkim:!:19000:0:99999:7:::' >> /etc/shadow
```

Phase 2 — Install Postfix

```
apt-get install -y postfix cyrus-sasl cyrus-sasl-lib
```

Fix queue directory permissions after installation:

```
postfix set-permissions
```

Enable Postfix logging:

```
touch /var/log/maillog
chmod 600 /var/log/maillog
postconf -e 'maillog_file = /var/log/maillog'
```

Replace the Postfix init.d Script

The init.d script that comes with the PCLinuxOS Postfix installation does not work correctly. Replace it with a minimal script that calls the native postfix command directly.

First back up the existing script:

```
mv /etc/init.d/postfix /etc/init.d/postfix.orig
```

Create the new script:

```
cat > /etc/init.d/postfix << 'EOF'
#!/bin/bash
# Postfix SMTP relay
# chkconfig: 2345 80 30
# description: Postfix SMTP relay with Gmail OAuth2

case "$1" in
    start|stop|restart|reload|abort|flush|check|status)
        /usr/sbin/postfix "$1"
        ;;
    *)
        echo "Usage: $0 {start|stop|restart|reload|abort|flush|check|status}"
        exit 1
        ;;
esac
EOF

chmod 755 /etc/init.d/postfix
```

Register with chkconfig and enable on boot:

```
chkconfig --add postfix
chkconfig postfix on
```

Verify it is enabled in the correct runlevels:

```
chkconfig --list postfix
```

Test the script:

```
/etc/init.d/postfix stop
/etc/init.d/postfix start
/etc/init.d/postfix status
```

Phase 3 — Register a Google OAuth2 App

3.1 Create the Project and Credentials

- Go to console.cloud.google.com
- Create a new project or select an existing one
- Enable the Gmail API under APIs & Services
- Go to Credentials → Create Credentials → OAuth client ID
- Application type: Desktop app
- Download the JSON file — you will need the `client_id` and `client_secret`

3.2 Configure the OAuth Consent Screen

- Go to APIs & Services → OAuth consent screen

- Choose External as the user type
- Fill in the required app name and contact fields
- Go to Verification Center → View Verification Audience
- Click on Audience in left Menu (that just appeared)
- Add your Gmail address as a test user
- Click on Publish App (publishing status to 'In production')

3.3 Get the Initial Refresh Token

Run `get_new_refresh_token.sh` on a machine with a browser. It will open a browser window for Google login and print the refresh token to the terminal.

```
bash get_new_refresh_token.sh
```

Save the refresh token — then run `set_refresh_token.sh` to write it to the token files:

```
bash set_refresh_token.sh youraddress@gmail.com <REFRESH_TOKEN>
```

Phase 4 — Build and Install sasl-xoauth2

Install build dependencies:

```
apt-get install -y git cmake libcurl-devel jsoncpp-devel cyrus-sasl-devel gcc-c++
```

Build and install the plugin:

```
git clone https://github.com/tarickb/sasl-xoauth2.git
cd sasl-xoauth2
mkdir build && cd build
cmake -DCMAKE_INSTALL_PREFIX=/usr -DBUILD_MANPAGES=OFF ..
make
make install
```

Verify the plugin was installed:

```
ls /usr/lib64/sasl2/libsasl-xoauth2.so
```

Phase 5 — Configure sasl-xoauth2

5.1 Plugin config file

```
cat > /etc/sasl-xoauth2.conf << 'EOF'
{
  "client_id": "YOUR_CLIENT_ID",
  "client_secret": "YOUR_CLIENT_SECRET",
  "log_to_syslog_on_failure": "yes",
  "log_full_trace_on_failure": "yes"
}
EOF

chown root:postfix /etc/sasl-xoauth2.conf
chmod 640 /etc/sasl-xoauth2.conf

# Also copy into Postfix directory
cp /etc/sasl-xoauth2.conf /var/spool/postfix/etc/sasl-xoauth2.conf
chown root:postfix /var/spool/postfix/etc/sasl-xoauth2.conf
chmod 640 /var/spool/postfix/etc/sasl-xoauth2.conf
```

5.2 SASL config

Add xoauth2 to the SASL plugin config:

```
cat >> /etc/sasl2/smtpd.conf << 'EOF'
pwcheck_method: auxprop
auxprop_plugin: xoauth2
EOF
```

5.3 Token file

Create the token file with your refresh token. The plugin will automatically obtain and refresh the access token:

```
mkdir -p /etc/tokens

cat > /etc/tokens/youraddress@gmail.com << 'EOF'
{
  "access_token": "",
  "expiry": "0",
  "refresh_token": "YOUR_REFRESH_TOKEN"
}
EOF

chown postfix:postfix /etc/tokens/youraddress@gmail.com
chmod 660 /etc/tokens/youraddress@gmail.com
chown postfix:postfix /etc/tokens
chmod 770 /etc/tokens
```

5.4 SASL password map

```
echo '[smtp.gmail.com]:587 youraddress@gmail.com:/etc/tokens/youraddress@gmail.com' \
> /etc/postfix/sasl_passwd

postmap /etc/postfix/sasl_passwd
chown root:postfix /etc/postfix/sasl_passwd /etc/postfix/sasl_passwd.db
chmod 640 /etc/postfix/sasl_passwd /etc/postfix/sasl_passwd.db
```

Phase 6 — DKIM Setup

6.1 Install perl-Mail-DKIM

```
apt-get install -y perl-Mail-DKIM
```

6.2 Generate DKIM key pair

```
mkdir -p /etc/dkim/keys/yourdomain.tld
openssl genrsa -out /etc/dkim/keys/yourdomain.tld/mail.private 2048
openssl rsa -in /etc/dkim/keys/yourdomain.tld/mail.private -pubout \
-out /etc/dkim/keys/yourdomain.tld/mail.public

# Set permissions
chown root:dkim /etc/dkim
chmod 755 /etc/dkim
chown root:dkim /etc/dkim/keys
chmod 750 /etc/dkim/keys
chown root:dkim /etc/dkim/keys/yourdomain.tld
chmod 750 /etc/dkim/keys/yourdomain.tld
chown root:dkim /etc/dkim/keys/yourdomain.tld/mail.private
chmod 640 /etc/dkim/keys/yourdomain.tld/mail.private
chown root:dkim /etc/dkim/keys/yourdomain.tld/mail.public
chmod 644 /etc/dkim/keys/yourdomain.tld/mail.public
```

6.3 Add DKIM DNS TXT record

Extract the public key for DNS:

```
grep -v '^-' /etc/dkim/keys/yourdomain.tld/mail.public | tr -d '\n'
```

Add a DNS TXT record at your DNS provider with these values:

- Name/Host: mail._domainkey.yourdomain.com
- Type: TXT
- Value: v=DKIM1; k=rsa; p=YOUR_BASE64_PUBLIC_KEY

Verify the record has propagated:

```
dig TXT mail._domainkey.yourdomain.com +short
```

6.4 Add SPF DNS TXT record

Add a TXT record for your domain to authorize Gmail's servers to send on your behalf:

- Name/Host: yourdomain.com
- Type: TXT
- Value: v=spf1 include:_spf.google.com ~all

6.5 Create the DKIM signing script

```
cat > /usr/local/bin/dkim-sign.pl << 'EOF'
#!/usr/bin/perl
use strict;
use warnings;
use Mail::DKIM::Signer;
use Net::SMTP;

my ($sender, @recipients) = @ARGV;

my $dkim = Mail::DKIM::Signer->new(
    Algorithm => 'rsa-sha256',
    Method    => 'relaxed/relaxed',
    Domain    => 'yourdomain.tld',
    Selector  => 'mail',
    KeyFile   => '/etc/dkim/keys/yourdomain.tld/mail.private',
);

my @lines;
while (<STDIN>) {
    s/\r?\n$/r\n/;
    push @lines, $_;
    \ $dkim->PRINT(\$_);
}
\ $dkim->CLOSE();

my \ $signed = \ $dkim->signature->as_string() . "\r\n" . join('', @lines);

my \ $smtp = Net::SMTP->new('127.0.0.1', Port => 10026, Timeout => 30)
    or die "Cannot connect to localhost:10026: \ $!";

\ $smtp->mail(\ $sender);
\ $smtp->to(@recipients);
\ $smtp->data(\ $signed);
\ $smtp->quit();
EOF

chown root:dkim /usr/local/bin/dkim-sign.pl
chmod 750 /usr/local/bin/dkim-sign.pl
```

Phase 7 — Configure Postfix

7.1 main.cf changes

Comment out the original `inet_interfaces`, `inet_protocols`, and `smtp_tls_security_level` lines, then add the following at the bottom of `/etc/postfix/main.cf`:

```
# Relay to Gmail
relayhost = [smtp.gmail.com]:587

# Require TLS to Gmail
smtp_tls_security_level = encrypt
smtp_tls_CAfile = /etc/pki/tls/certs/ca-bundle.crt

# SASL auth using XOAUTH2
smtp_sasl_auth_enable = yes
smtp_sasl_mechanism_filter = xoauth2
smtp_sasl_security_options =
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd

# Trusted networks – localhost and your mail server IPs
mynetworks = 127.0.0.0/8 <mail_server_ip_address>/32

smtpd_relay_restrictions = permit_mynetworks, reject
smtpd_recipient_restrictions = permit_mynetworks, reject

# Listen on all interfaces, IPv4 only
inet_interfaces = all
inet_protocols = ipv4

# Logging
maillog_file = /var/log/maillog

# DKIM content filter
content_filter = dkimfilter:dummy
```

7.2 master.cf changes

Make the following changes to `/etc/postfix/master.cf`:

1. Change the inbound smtp listener from port 25 to port 2525 (XMail uses port 25):

2525	inet	n	-	y	-	-	smtpd
------	------	---	---	---	---	---	-------

2. Disable chroot for outbound smtp and relay transports (change 'y' to 'n'):

smtp	unix	-	-	n	-	-	smtp
relay	unix	-	-	n	-	-	smtp

3. Add the DKIM filter transport at the end of master.cf:

```
dkimfilter unix      -    n    n    -    -    pipe
                   flags=Rq user=dkim argv=/usr/bin/perl /usr/local/bin/dkim-sign.pl ${sender} ${recipient}
```

Note: The 10026 re-injection port is already defined in the default master.cf. Do not add it again.

Phase 8 — Start and Test Postfix

```
postfix start

# Verify listening on port 2525
ss -tlnp | grep 2525

# Send a test email
echo 'Test from relay' | sendmail youraddress@gmail.com

# Watch the logs
tail -f /var/log/maillog
```

A successful delivery will show:

```
status=sent (250 2.0.0 OK)
```

Check the received email headers to verify DKIM is present:

```
dkim=pass header.d=yourdomain.tld
```

Phase 9 — Configure XMail

9.1 Edit smtpfwd.tab

Update the routing table to send outbound mail through the Postfix relay. Edit the file in your XMail data directory:

```
"gmail.com"      "127.0.0.1:2525"
"outlook.com"    "127.0.0.1:2525"
```

Note: Add a line for any other domain that requires authenticated SMTP delivery.

9.2 Remove old Gmail credentials

Remove the old XMail SMTP authentication file for Gmail — it is no longer needed:

```
rm /path/to/xmail/userauth/smtp/smtp.gmail.com.tab
```

9.3 Restart XMail

Restart XMail to apply the routing changes.

Phase 10 — Enable Postfix on Boot

```
chkconfig postfix on
```

Set up `update_refresh_token.sh` as a cron job to run every 5 days. Add to `/etc/anacrontab`

```
echo "5    10    gmail-token-refresh    bash /path/to/update_refresh_token.sh  
youraddress@gmail.com >> /var/log/gmail-token-refresh.log 2>&1" >> /etc/anacrontab
```

Security Notes

- `mynetworks` is your trust boundary — only IPs listed there can relay through Postfix without authentication. Keep it restricted to `localhost` and your mail server IPs only.
- The OAuth2 token file is owned by `postfix:postfix` with permissions 660. The tokens directory has permissions 770.
- The DKIM private key is owned by `root:dkim` with permissions 640. Only the `dkim` user can read it.
- The `sasl-xoauth2.conf` containing `client_id` and `client_secret` is owned by `root:postfix` with permissions 640.
- OAuth2 access tokens expire every hour. The `sasl-xoauth2` plugin refreshes them automatically using the refresh token.
- Postfix worker processes run as the `postfix` user, not `root`. The DKIM filter runs as the dedicated `dkim` user.
- Chroot is disabled for outbound smtp processes. This is necessary for the `sasl-xoauth2` plugin to find its config and token files.
- Set the Google OAuth app publishing status to 'In production'.

Troubleshooting

'No worthy mechs found'

The `sasl-xoauth2` plugin is not being loaded. Check:

- `/usr/lib64/sasl2/libsasl-xoauth2.so` exists
- `/etc/sasl2/smtpd.conf` contains `pwcheck_method: auxprop` and `auxprop_plugin: xoauth2`
- `/etc/sasl-xoauth2.conf` exists with valid `client_id` and `client_secret`
- `/var/spool/postfix/etc/sasl-xoauth2.conf` is a copy of `/etc/sasl-xoauth2.conf`

'Permission denied' on token file

```
chown postfix:postfix /etc/tokens/youraddress@gmail.com
chmod 660 /etc/tokens/youraddress@gmail.com
chown postfix:postfix /etc/tokens
chmod 770 /etc/tokens
```

'Address already in use' on port 25

Another service is using port 25. Ensure the inbound listener in master.cf is set to 2525.

'unknown user name: postfix'

The postfix system user was not created during installation. Follow Phase 1 to create it manually.

Token refresh fails with HTTP 400

The refresh token is invalid or has been revoked. First verify the token is actually invalid by running check_refresh_token.sh. A 400 response with error “invalid_grant” confirms revocation.

To fix, re-authorize and update both token files:

```
# Step 1: Get a new refresh token (requires a machine with a browser)
bash get_new_refresh_token.sh

# Step 2: Write the new token to both token files
bash set_refresh_token.sh youraddress@gmail.com <REFRESH_TOKEN>

# Step 3: Reload Postfix
postfix reload
```

DKIM verification fails

Check that:

- The DNS TXT record at mail._domainkey.yourdomain.com has propagated (use dig to verify)
- The domain and selector in dkim-sign.pl match the DNS record
- The private key file is readable by the dkim user
- The dkimfilter transport in master.cf is correctly defined